

Федеральное государственное автономное образовательное учреждение
высшего образования
«МОСКОВСКИЙ ПОЛИТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ»

Факультет Информационных Технологий
Кафедра «Автоматизированные системы обработки информации и управления»

Направление подготовки/ специальность: Бэкенд

ОТЧЕТ

по проектной практике

Студент: Тулин Тимофей Павлович, Группа: 251-331

Место прохождения практики: Московский Политех, кафедра
Автоматизированные системы обработки информации и управления

Отчет принят с оценкой _____ Дата _____

Руководитель практики: _____

Москва 2025

ОГЛАВЛЕНИЕ

Введение

1. Общая информация о проекте

1.1. Название проекта

1.2. Цели и задачи проекта

1.3. Общая характеристика деятельности организации

2. Описание задания по проектной практике

2.1. Индивидуальная задача по проекту NeoPixel

2.2. Статический сайт по проектной деятельности

3. Вариативная часть практики

3.1. Постановка задачи

3.2. Реализация простого HTTP-сервера

3.3. Проверка результата

4. Описание достигнутых результатов

Заключение

Список использованной литературы

Приложения

ВВЕДЕНИЕ

Проектная практика была выполнена в рамках учебной деятельности Московского Политехнического университета. Работа включала две части: подготовку статического сайта по проектной деятельности и выполнение индивидуальной вариативной части по выбранной теме из открытого репозитория build-your-own-х.

Основная проектная часть связана с проектом NeoPixel - учебным проектом по разработке и доработке малого фотополимерного 3D-принтера. В рамках этой части была выбрана техническая задача по анализу программной логики управления осью Z. Вариативная часть была выполнена индивидуально и посвящена разработке простого HTTP-сервера на Python.

1. Общая информация о проекте

1.1. Название проекта

Название проектной деятельности: NeoPixel. Проект связан с фотополимерной 3D-печатью и разработкой учебного прототипа принтера.

1.2. Цели и задачи проекта

Цель проектной части - разобраться в устройстве проекта NeoPixel и подготовить понятное представление одной из его технических задач на статическом сайте.

- изучить структуру репозитория Mini_Pixel_proj и материалы по проекту;
- выбрать индивидуальную техническую задачу, связанную с программной логикой принтера;
- разобрать назначение основных функций управления осью Z;
- подготовить блок-схему кодовой логики;
- оформить результаты в виде статического сайта и сопроводительной документации.

1.3. Общая характеристика деятельности организации

Практика выполнялась в образовательной среде Московского Политехнического университета. Заказчиком выступает учебное подразделение, организующее проектную деятельность и практическую подготовку студентов. Деятельность ориентирована на получение прикладных навыков: работу с репозиториями, документацией, сайтами, техническим описанием и программной реализацией.

Участники, отраженные на сайте

Участник	Краткий вклад
Тулин Тимофей Павлович	Анализ подключения A4988 и кодовой логики управления осью Z.
Репина Алёна	Подготовка презентационных материалов и демонстрация проекта.
Попов В. С.	Анализ вариантов GUI с учетом ограничений ESP.
Мачигин Т. П.	Обсуждение модификаций корпуса и оси Z.
Донской С. Д.	Работа с моделями, пайкой и порядком в лаборатории.
Кузьмин А. Ю.	Работа с моделями, обслуживанием станка и материалами.

2. Описание задания по проектной практике

2.1. Индивидуальная задача по проекту NeoPixel

В качестве индивидуальной задачи была выбрана тема анализа кодовой логики управления осью Z фотополимерного 3D-принтера NeoPixel. В репозитории проекта рассматривалась логика класса StepMotor, связанная с ESP32-S3, драйвером A4988, шаговым двигателем NEMA17 и концевым выключателем.

Задача заключалась не в разработке нового модуля прошивки, а в разборе существующей логики и ее объяснении в понятном виде: какие функции участвуют в движении платформы, как формируются STEP-импульсы, как задается направление и как выполняется поиск нулевой позиции.

Элемент	Назначение
app_main()	точка входа программы ESP-IDF
StepMotor	создание объекта управления шаговым двигателем
begin()	настройка GPIO и параметров
enable()	активация драйвера A4988
homing()	поиск нулевой позиции по концевiku
move()	генерация STEP-импульсов
moveMM()	перевод миллиметров в шаги
printLayerSequence()	последовательность подъема, возврата и движения слоя

Этапы выполнения задачи

1. v0: Изучены README.md, src/main.cpp и include/Stepmotor.h; выбрана задача по оси Z.
2. v1: Разобрано подключение драйвера A4988 и сигналы STEP, DIR, EN.
3. v2: Описана логика StepMotor: настройка GPIO, включение драйвера, движение и учет позиции.
4. v3: Отдельно рассмотрены homing, работа концевика и безопасный поиск нулевой позиции.
5. v4: Результат оформлен на сайте, добавлена блок-схема кодовой логики и пояснение этапов.

2.2. Статический сайт по проектной деятельности

Статический сайт расположен в папке site/. Он выполнен как самостоятельная HTML-страница с отдельным CSS-файлом и изображением блок-схемы в папке images/. На сайте представлены описание проекта NeoPixel, участники, индивидуальная задача, этапы выполнения и итоговая схема кодовой работы.



Рисунок 1 - Блок-схема кодовой логики управления осью Z

3. Вариативная часть практики

3.1. Постановка задачи

Для вариативной части была выбрана тема из репозитория `codecrafters-io/build-your-own-x`: Build your own Web Server / Python: A Simple Web Server. Задача выполнялась индивидуально и была размещена в папке `src/`.

Цель вариативной части - изучить базовый цикл работы HTTP-сервера без использования Flask, Django, FastAPI и других готовых веб-фреймворков.

3.2. Реализация простого HTTP-сервера

Реализация выполнена на стандартной библиотеке Python. Проект разделен на файл запуска `main.py`, файл основной логики `server.py` и папку `public/` со статическими HTML- и CSS-файлами.

- открытие TCP-сокета и прием HTTP-запроса;
- разбор метода и пути запроса;
- маршруты `/` и `/about`;
- выдача CSS-файла через `/static/style.css`;
- обработка ошибок 404 Not Found и 405 Method Not Allowed;
- консольное логирование запросов.

3.3. Проверка результата

Проверка	Ожидаемый код	Результат
/	200	главная страница открывается
/about	200	страница описания открывается
/static/style.css	200	CSS-файл отдается сервером
/missing	404	неизвестный путь обрабатывается
POST /	405	неподдерживаемый метод отклоняется

4. Описание достигнутых результатов

- создана структура репозитория практики по требованиям методических материалов;
- подготовлен статический сайт по проектной деятельности NeoPixel;
- описана индивидуальная задача по анализу управления осью Z;
- добавлена блок-схема кодовой логики;
- реализован простой HTTP-сервер на Python для вариативной части;
- подготовлены Markdown-файлы с документацией, журналом и техническим руководством.

ЗАКЛЮЧЕНИЕ

В ходе практики были оформлены материалы по проектной деятельности и выполнена индивидуальная вариативная задача. Работа позволила закрепить навыки ведения репозитория, подготовки статического сайта, технического описания проекта и реализации сетевого приложения на базовом уровне.

Ценность выполненных задач для заказчика заключается в том, что результаты практики представлены в понятном и проверяемом виде: сайт демонстрирует проектную часть, документация фиксирует этапы работы, а исходный код вариативной части показывает понимание принципов HTTP-сервера.

СПИСОК ИСПОЛЬЗОВАННОЙ ЛИТЕРАТУРЫ

6. Репозиторий методических материалов по проектной практике:
<https://github.com/mospol/practice-2025-1>
7. Задание по структуре репозитория и сайту практики:
<https://github.com/mospol/practice-2025-1/blob/master/task/README.md>
8. Репозиторий с темами для вариативной части build-your-own-x:
<https://github.com/codecrafters-io/build-your-own-x>
9. Раздел Build your own Web Server в build-your-own-x:
<https://github.com/codecrafters-io/build-your-own-x#build-your-own-web-server>
10. Учебный репозиторий проекта NeoPixel:
https://github.com/MEISSSTER/Mini_Pixel_proj
11. Итоговый репозиторий практики Тулина Т. П.:
<https://github.com/eyeless03/practice-2026>
12. Документация Python. Модуль socket:
<https://docs.python.org/3/library/socket.html>
13. Документация Python. Модуль argparse:
<https://docs.python.org/3/library/argparse.html>
14. Документация Python. Модуль pathlib:
<https://docs.python.org/3/library/pathlib.html>
15. Документация Python. Модуль mimetypes:
<https://docs.python.org/3/library/mimetypes.html>
16. Документация Python. Модуль http.server:
<https://docs.python.org/3/library/http.server.html>
17. MDN Web Docs. Обзор протокола HTTP:
<https://developer.mozilla.org/docs/Web/HTTP/Guides/Overview>
18. MDN Web Docs. HTTP request methods:
<https://developer.mozilla.org/docs/Web/HTTP/Reference/Methods>
19. MDN Web Docs. HTTP response status codes:
<https://developer.mozilla.org/docs/Web/HTTP/Reference/Status>
20. MDN Web Docs. MIME types:
https://developer.mozilla.org/docs/Web/HTTP/Guides/MIME_types
21. Espressif ESP-IDF Programming Guide. GPIO & RTC GPIO:
<https://docs.espressif.com/projects/esp-idf/en/stable/esp32/api-reference/peripherals/gpio.html>

22. Espressif ESP-IDF Programming Guide. FreeRTOS:
<https://docs.espressif.com/projects/esp-idf/en/stable/esp32/api-reference/system/freertos.html>
23. Pololu. A4988 Stepper Motor Driver Carrier:
<https://www.pololu.com/product/1182>
24. Git Documentation: <https://git-scm.com/doc>
25. GitHub Docs. About repositories:
<https://docs.github.com/en/repositories/creating-and-managing-repositories/about-repositories>

ПРИЛОЖЕНИЯ

Приложение А. Структура репозитория: README.md, docs/, site/, src/, reports/. Команда запуска вариативной части: `python src/main.py --host 127.0.0.1 --port 8080`.

Ниже приведены листинги исходного кода вариативной части, в которой реализован простой HTTP-сервер на Python.

Приложение Б. Листинг файла `main.py`

Файл: `src/main.py`

```
001 from __future__ import annotations
002
003 import argparse
004
005 from server import SimpleHTTPServer
006
007
008 def parse_args() -> argparse.Namespace:
009     parser = argparse.ArgumentParser(description="Simple educational HTTP server")
010     parser.add_argument("--host", default="127.0.0.1", help="Host to bind")
011     parser.add_argument("--port", default=8080, type=int, help="Port to listen on")
012     return parser.parse_args()
013
014
015 def main() -> None:
016     args = parse_args()
017     server = SimpleHTTPServer(host=args.host, port=args.port)
018     server.serve_forever()
019
020
021 if __name__ == "__main__":
022     main()
```

Приложение В. Листинг файла `server.py`

Файл: `src/server.py`

```
001 from __future__ import annotations
002
003 import mimetypes
```

```

004 import socket
005 from dataclasses import dataclass
006 from datetime import datetime
007 from pathlib import Path
008 from typing import Callable
009 from urllib.parse import unquote
010
011
012 CRLF = "\r\n"
013
014
015 @dataclass
016 class Request:
017     method: str
018     path: str
019     version: str
020
021
022 @dataclass
023 class Response:
024     status: int
025     reason: str
026     body: bytes
027     content_type: str = "text/html; charset=utf-8"
028
029
030 class SimpleHTTPServer:
031     def __init__(self, host: str = "127.0.0.1", port: int = 8080) -> None:
032         self.host = host
033         self.port = port
034         self.public_dir = Path(__file__).resolve().parent / "public"
035         self.routes: dict[str, Callable[[], Response]] = {
036             "/": lambda: self._html_file("index.html"),
037             "/about": lambda: self._html_file("about.html"),
038         }
039
040     def serve_forever(self) -> None:
041         with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as server_socket:
042             server_socket.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
043             server_socket.bind((self.host, self.port))
044             server_socket.listen()
045
046             print(f"Server started on http://{self.host}:{self.port}")
047             print("Press Ctrl+C to stop.")
048
049             while True:
050                 client_socket, client_address = server_socket.accept()
051                 with client_socket:
052                     self._handle_client(client_socket, client_address[0])
053
054     def _handle_client(self, client_socket: socket.socket, client_ip: str) -> None:
055         raw_request = client_socket.recv(4096)
056         request = self._parse_request(raw_request)
057
058         if request is None:
059             response = self._plain_text(400, "Bad Request", "Bad Request")
060             log_method = "-"
061             log_path = "-"
062         elif request.method != "GET":
063             response = self._plain_text(405, "Method Not Allowed", "Method Not Allowed")
064             log_method = request.method
065             log_path = request.path
066         else:
067             response = self._route(request.path)
068             log_method = request.method
069             log_path = request.path
070
071         client_socket.sendall(self._build_response(response))
072         self._log(client_ip, log_method, log_path, response.status)
073
074     def _parse_request(self, raw_request: bytes) -> Request | None:
075         try:
076             first_line = raw_request.decode("iso-8859-1").splitlines()[0]
077             method, path, version = first_line.split()
078         except (IndexError, ValueError, UnicodeDecodeError):
079             return None
080
081         return Request(method=method.upper(), path=path, version=version)
082

```

```

083     def _route(self, raw_path: str) -> Response:
084         path = unquote(raw_path.split("?", 1)[0])
085
086         if path in self.routes:
087             return self.routes[path]()
088
089         if path.startswith("/static/"):
090             relative_path = path.removeprefix("/static/")
091             return self._static_file(relative_path)
092
093         return self._html_file("404.html", status=404, reason="Not Found")
094
095     def _html_file(self, filename: str, status: int = 200, reason: str = "OK") ->
Response:
096         file_path = self.public_dir / filename
097
098         if not file_path.exists():
099             return self._plain_text(500, "Internal Server Error", f"Missing file:
{filename}")
100
101         return Response(
102             status=status,
103             reason=reason,
104             body=file_path.read_bytes(),
105             content_type="text/html; charset=utf-8",
106         )
107
108     def _static_file(self, relative_path: str) -> Response:
109         requested = (self.public_dir / relative_path).resolve()
110
111         if not self._is_inside_public(requested) or not requested.is_file():
112             return self._html_file("404.html", status=404, reason="Not Found")
113
114         content_type, _ = mimetypes.guess_type(requested.name)
115         return Response(
116             status=200,
117             reason="OK",
118             body=requested.read_bytes(),
119             content_type=content_type or "application/octet-stream",
120         )
121
122     def _is_inside_public(self, file_path: Path) -> bool:
123         public_root = self.public_dir.resolve()
124         return file_path == public_root or public_root in file_path.parents
125
126     def _plain_text(self, status: int, reason: str, message: str) -> Response:
127         return Response(
128             status=status,
129             reason=reason,
130             body=message.encode("utf-8"),
131             content_type="text/plain; charset=utf-8",
132         )
133
134     def _build_response(self, response: Response) -> bytes:
135         headers = [
136             f"HTTP/1.1 {response.status} {response.reason}",
137             f"Content-Type: {response.content_type}",
138             f"Content-Length: {len(response.body)}",
139             "Connection: close",
140             "",
141             "",
142         ]
143         return CRLF.join(headers).encode("utf-8") + response.body
144
145     def _log(self, client_ip: str, method: str, path: str, status: int) -> None:
146         now = datetime.now().strftime("%Y-%m-%d %H:%M:%S")
147         print(f"{now} {client_ip} {method} {path} {status}")

```

Приложение Г. Листинг файла public/index.html

Файл: *src/public/index.html*

```

001 <!doctype html>
002 <html lang="ru">
003     <head>
004         <meta charset="utf-8">

```

```

005     <title>Простой HTTP-сервер</title>
006     <link rel="stylesheet" href="/static/style.css">
007 </head>
008 <body>
009     <main class="page">
010         <p class="eyebrow">Вариативная часть практики</p>
011         <h1>Простой HTTP-сервер на Python</h1>
012         <p>
013             Это учебный сервер, написанный без веб-фреймворков. Он показывает,
014             как Python принимает HTTP-запрос, выбирает маршрут и формирует ответ.
015         </p>
016         <nav>
017             <a href="/">Главная</a>
018             <a href="/about">О проекте</a>
019             <a href="/missing">Проверка 404</a>
020         </nav>
021     </main>
022 </body>
023 </html>

```

Приложение Д. Листинг файла public/about.html

Файл: *src/public/about.html*

```

001 <!doctype html>
002 <html lang="ru">
003     <head>
004         <meta charset="utf-8">
005         <title>О проекте</title>
006         <link rel="stylesheet" href="/static/style.css">
007     </head>
008     <body>
009         <main class="page">
010             <p class="eyebrow">О проекте</p>
011             <h1>Как работает сервер</h1>
012             <p>
013                 Сервер открывает TCP-сокеты, ожидает подключение клиента, читает первую
014                 строку HTTP-запроса и выбирает обработчик по пути запроса.
015             </p>
016             <ul>
017                 <li><code>GET /</code> возвращает главную страницу.</li>
018                 <li><code>GET /about</code> возвращает описание проекта.</li>
019                 <li><code>GET /static/style.css</code> возвращает CSS-файл.</li>
020                 <li>Неизвестный адрес возвращает <code>404 Not Found</code>.</li>
021             </ul>
022             <nav>
023                 <a href="/">На главную</a>
024             </nav>
025         </main>
026     </body>
027 </html>

```

Приложение Е. Листинг файла public/404.html

Файл: *src/public/404.html*

```

001 <!doctype html>
002 <html lang="ru">
003     <head>
004         <meta charset="utf-8">
005         <title>404 Not Found</title>
006         <link rel="stylesheet" href="/static/style.css">
007     </head>
008     <body>
009         <main class="page">
010             <p class="eyebrow">404</p>
011             <h1>Страница не найдена</h1>
012             <p>Сервер получил запрос, но не нашел подходящий маршрут или файл.</p>
013             <nav>
014                 <a href="/">Вернуться на главную</a>
015             </nav>
016         </main>

```

```
017 </body>
018 </html>
```

Приложение Ж. Листинг файла public/style.css

Файл: src/public/style.css

```
001 body {
002     margin: 0;
003     min-height: 100vh;
004     display: grid;
005     place-items: center;
006     font-family: Arial, Helvetica, sans-serif;
007     color: #111827;
008     background: linear-gradient(135deg, #edf5ff, #f8fbff);
009 }
010
011 .page {
012     width: min(760px, calc(100% - 40px));
013     padding: 42px;
014     background: #ffffff;
015     border: 1px solid #d8dee8;
016     border-radius: 8px;
017     box-shadow: 0 18px 45px rgba(20, 28, 45, 0.12);
018 }
019
020 .eyebrow {
021     margin: 0 0 12px;
022     color: #2563eb;
023     font-size: 14px;
024     font-weight: 700;
025     text-transform: uppercase;
026 }
027
028 h1 {
029     margin: 0 0 16px;
030     font-size: 38px;
031     line-height: 1.1;
032 }
033
034 p,
035 li {
036     color: #4b5565;
037     font-size: 18px;
038     line-height: 1.6;
039 }
040
041 nav {
042     display: flex;
043     flex-wrap: wrap;
044     gap: 12px;
045     margin-top: 26px;
046 }
047
048 a {
049     display: inline-flex;
050     padding: 10px 14px;
051     color: #ffffff;
052     background: #2563eb;
053     border-radius: 8px;
054     text-decoration: none;
055 }
056
057 code {
058     color: #0f2a5f;
059     background: #edf5ff;
060     padding: 2px 6px;
061     border-radius: 5px;
062 }
```